

Protocol-Based Liability Attribution in Semi-Autonomous Industrial Systems: The Sopcos Architecture

Ekrem ORAL (ekremoral@gmail.com)

2026

Abstract

As Industrial IoT (IIoT) systems evolve toward semi-autonomous operation, a critical "Liability Gap" emerges: while control systems such as SCADA record telemetry, they fail to cryptographically bind decisions to specific authorities, making forensic attribution subjective, disputable, and computationally infeasible. This paper introduces the Sopcos Protocol, a deterministic layer-1 liability architecture that transforms operational decisions into cryptographically verifiable artifacts.

The architecture consists of three core components: an immutable identity and policy ledger, a neutral execution courier, and a deterministic adjudication runtime. Together, these components produce reproducible verdicts that formally attribute each action to its originating authority.

We formalize decision resolution through a Verdict Algebra, which deterministically resolves conflicts between algorithmic policies, human interventions, and physical safety constraints in time linear in the number of candidate verdicts $O(|V_x|)$. The system enforces a Ring Protection Model, ensuring that physical invariants supersede human overrides, and human authority supersedes algorithmic optimization. Additionally, we introduce the Cognitive Verdict mechanism, which confines probabilistic AI systems to advisory roles by requiring deterministic policy validation before any safety-critical execution.

We validate the architecture using a canonical industrial incident scenario, demonstrating how Sopcos creates an immutable chain of custody for operational decisions. This enables deterministic replay, cryptographic attribution, and verifiable liability, transforming industrial incident analysis from subjective interpretation into objective protocol-level proof.

1 Introduction

The transition from Industry 4.0 to Industry 5.0 is characterized by a fundamental shift from monitoring to autonomy. While previous industrial paradigms focused on interconnectivity, telemetry collection, and supervisory control through Industrial Control Systems (ICS) and Supervisory Control and Data Acquisition (SCADA), emerging industrial environments increasingly delegate safety-critical decision-making to algorithmic agents, semi-autonomous controllers, and advisory artificial intelligence systems. This evolution transforms industrial infrastructure from passive monitoring systems into active decision-making entities.

However, this transition introduces a critical architectural and legal challenge: the **Industrial Liability Gap**.

Traditional ICS and SCADA architectures operate on a model of implicit trust. They are designed to record operational events — such as sensor readings, actuator commands, and operator actions. But they are structurally incapable of proving, with cryptographic certainty, why a particular action occurred or which authority authorized it. Operational logs can indicate that a valve opened at a specific time, but they cannot deterministically establish whether the action resulted from deterministic control logic, human intervention, automated optimization, or advisory AI influence. As a result, causal attribution in post-incident forensic analysis becomes inherently interpretive, relying on correlating fragmented logs, system configurations, and operator records.

In complex cyber-physical systems where multiple authorities coexist — including physical safety constraints, deterministic control policies, human operators, and probabilistic AI models — the absence of deterministic attribution creates ambiguity in both operational accountability and legal responsibility. This ambiguity is particularly problematic in safety-critical infrastructure, where liability must be established with verifiable certainty rather than inferred through probabilistic reconstruction.

We posit that autonomy requires a new class of infrastructure: not merely for execution, but for **adjudication**.

Execution determines what action is performed. Adjudication determines whether that action is authorized, valid, and attributable. Existing industrial architectures perform execution but lack a deterministic, cryptographically verifiable adjudication layer. Without such a layer, industrial systems cannot produce authoritative proof linking decisions to specific identities, policies, or governing constraints.

To address this problem, we introduce the concept of **protocol-based liability attribution**, in which decision authority, policy resolution, and execution provenance are enforced through a deterministic protocol rather than implicit system configuration.

This paper presents the **Sopcos Protocol**, a deterministic liability-layer architecture designed to transform industrial decisions into cryptographically verifiable protocol artifacts. The protocol is built on three foundational principles:

- **Deterministic adjudication**, ensuring that identical inputs always produce identical verdicts
- **Cryptographic authority binding**, ensuring that each decision can be attributed to a specific identity and governing policy
- **Immutable decision provenance**, ensuring that adjudication results can be independently verified and replayed

Unlike general-purpose distributed ledgers designed for transaction processing or arbitrary computation, Sopcos implements a **Thin Ledger** architecture focused exclusively on anchoring identity, policy, and adjudication artifacts. The protocol explicitly separates the **Decision** (the adjudicated verdict) from the **Action** (the physical actuation), establishing a cryptographically verifiable chain of custody for industrial intent.

The Sopcos architecture consists of three logically distinct components:

- (1) an immutable identity and policy ledger that anchors authority and execution artifacts,
- (2) a neutral resolution and transport layer that retrieves and delivers adjudication context without interpretation, and
- (3) a deterministic adjudication runtime that executes policies in a hermetic environment and produces reproducible verdicts.

Together, these components transform operational records from passive logs into active cryptographic evidence. Under this model, each state transition in a cyber-physical system can be deterministically traced to a specific authority, policy, and adjudication result.

A central contribution of this work is the formalization of a **Verdict Algebra**, which defines deterministic conflict resolution between heterogeneous authorities, including physical constraints, regulatory policy, human intervention, and algorithmic control logic. This model is reinforced by a hierarchical authority framework, referred to as the **Ring Protection Model**, which enforces precedence relationships that prevent higher-level optimization or human override from violating fundamental safety constraints.

Additionally, we introduce the **Cognitive Verdict mechanism**, which confines probabilistic artificial intelligence systems to advisory roles. Under this model, AI-generated recommendations cannot directly trigger safety-critical actions without deterministic protocol validation, preserving attribution integrity and preventing non-deterministic execution authority.

We present Sopcos both as a protocol model and as a reference implementation demonstrating the feasibility of deterministic liability attribution in semi-autonomous industrial systems. Through a canonical incident scenario, we demonstrate how the protocol produces immutable adjudication artifacts that enable deterministic replay, independent verification, and cryptographic attribution of operational decisions.

The contributions of this paper are as follows:

1. We formally define the Industrial Liability Gap as an architectural limitation of existing industrial control systems.
2. We introduce protocol-based liability attribution as a deterministic alternative to traditional log-based forensic reconstruction.
3. We formalize a Verdict Algebra and hierarchical authority model for deterministic conflict resolution in cyber-physical systems.
4. We present the Sopcos Protocol as both a protocol model and reference implementation enabling cryptographically verifiable decision attribution.
5. We demonstrate how deterministic adjudication and immutable provenance transform industrial incident analysis into a protocol-verifiable process.

By elevating liability attribution from an interpretive forensic activity to a deterministic protocol property, the Sopcos Protocol establishes a foundational framework for accountable autonomy in industrial environments.

2 Formal Problem Definition: The Industrial Liability Gap

Modern industrial control paradigms operate under the implicit assumption that telemetry logs constitute authoritative records of system behavior. We demonstrate that in semi-autonomous cyber-physical systems, this assumption is false. Specifically, existing architectures cannot deterministically and cryptographically attribute executed actions to their originating authorities. This structural limitation creates what we formally define as the **industrial liability gap**.

We begin by defining a formal system model and attribution function and then demonstrate how existing architectures fail to satisfy deterministic attribution due to epistemic opacity, adversarial compliance, and the integration of probabilistic advisory systems.

2.1 System Model

We model an industrial cyber-physical system as a tuple:

$$S = (\mathcal{X}, \mathcal{A}, \mathcal{U}, \mathcal{P}, \mathcal{E})$$

where:

- $\mathcal{X}$ is the set of system states
- $\mathcal{A}$ is the set of executable actions
- $\mathcal{U}$ is the set of authorities
- $\mathcal{P}$ is the set of policies

- $\mathcal{E}$ is the set of execution events

A system state $x \in \mathcal{X}$ represents the observable configuration of the system, including sensor readings, controller registers, and operational parameters.

An action $a \in \mathcal{A}$ represents a discrete command capable of modifying the system state, such as actuating a valve, changing a control register, or initiating a shutdown procedure.

An authority $u \in \mathcal{U}$ represents any entity capable of influencing system behavior. Authorities may include deterministic control policies, human operators, supervisory control systems, or advisory computational agents.

An execution event is defined as a state transition:

$$e = (x_t, a, x_{t+1})$$

indicating that action a was executed when the system was in state x_t , producing a new state x_{t+1} .

2.2 Decision, Action, and Authority Model

We distinguish between a **Decision** and an **Action**.

A Decision is an adjudicated authorization produced by an authority or policy:

$$d = (u, p, x, a)$$

indicating that authority u , via policy p , authorizes action a in state x . An Action is the physical execution of that authorization:

$$e = (x_t, a, x_{t+1})$$

In a properly attributable system, each executed action must be traceable to a unique decision and originating authority.

However, conventional industrial systems record only execution events, not adjudicated decisions. This structural separation creates ambiguity in causal attribution.

2.3 Attribution Function and Deterministic Liability

We define the attribution function:

$$\text{Attr} : \mathcal{E} \rightarrow \mathcal{U}$$

which maps an execution event to the authority responsible for its authorization. A system satisfies deterministic liability attribution if and only if:

$$\forall e \in \mathcal{E}, \exists! u \in \mathcal{U} \text{ such that } \text{Attr}(e) = u$$

where $\exists!$ denotes existence and uniqueness. Additionally, attribution must satisfy the following properties:

- **Determinism:** attribution produces identical results for identical execution records
- **Reproducibility:** independent observers can compute attribution from system records alone
- **Cryptographic Verifiability:** attribution cannot be forged or altered without detection

Existing industrial control architectures do not satisfy these conditions.

2.4 Epistemic Opacity in Existing Systems

Traditional SCADA and industrial logging systems are designed for observability, not attribution. They record execution events but do not record adjudicated decisions or cryptographic authority bindings.

This creates a condition we define as **Epistemic Opacity**: the inability of a system to deterministically establish the causal authority responsible for an observed action.

For example, a valve opening event may be consistent with multiple authorities:

- deterministic safety logic executing automatically,
- a human operator issuing a manual override,
- an automated optimization routine adjusting system parameters,
- or a malicious actor injecting a command through a compromised endpoint.

Because execution logs do not cryptographically bind actions to specific authorities or policies, attribution becomes an inference problem rather than a deterministic computation.

Formally, epistemic opacity exists when:

$$\exists e \in \mathcal{E}, \exists u_1, u_2 \in \mathcal{U}, u_1 \neq u_2$$

such that both authorities are consistent with observed system records.

2.5 Adversarial Compliance and the "Evil but Compliant" Axiom

A fundamental limitation of log-based attribution is that system logs assume truthful reporting by participating components. However, adversarial nodes may remain syntactically compliant with communication protocols while semantically misrepresenting system state.

We define the **Evil but Compliant Axiom**: A system component may produce protocol-compliant messages that are structurally valid but semantically false.

For example, an intermediary gateway may suppress a critical sensor reading and replace it with a safe value. The resulting log entries remain syntactically valid and indistinguishable from legitimate telemetry.

Because logs lack cryptographic binding to originating authority and decision context, such adversarial manipulation cannot be reliably detected or attributed.

This condition prevents deterministic reconstruction of authority, even when execution logs are complete.

2.6 The Probabilistic Determinism Paradox

The integration of artificial intelligence introduces probabilistic actors into deterministic safety-critical systems.

Machine learning models generate outputs based on statistical inference rather than deterministic logic. Their outputs may influence operational decisions without producing deterministic execution artifacts or cryptographically attributable authority.

This creates what we define as the **Probabilistic Determinism Paradox**: non-deterministic advisory systems influence deterministic physical processes without producing deterministic attribution artifacts.

As a result, when a safety-critical action is influenced by probabilistic inference, existing architectures cannot establish verifiable attribution.

This creates a condition we define as a **Liability Vacuum**, in which no authority can be deterministically and cryptographically identified as responsible for the action.

2.7 Formal Definition of the Industrial Liability Gap

We define the Industrial Liability Gap as the condition under which deterministic attribution cannot be established.

Formally, the Liability Gap exists if:

$$\exists e \in \mathcal{E} \text{ such that } \text{Attr}(e)$$

is:

- non-unique, or
- non-deterministic, or
- non-verifiable

Under these conditions, attribution becomes an interpretive forensic process rather than a deterministic computation.

As a result, liability cannot be established as a protocol-level property of the system.

2.8 Requirements for Protocol-Level Liability Attribution

To eliminate the Industrial Liability Gap, attribution must be elevated from a post-hoc forensic inference to a protocol-enforced property. This requires a system in which:

- authority is cryptographically bound to decisions,
- decisions produce deterministic adjudication artifacts,
- adjudication artifacts are immutable,
- and attribution can be deterministically computed and independently verified.

In the following section, we introduce a theoretical framework that enables deterministic liability attribution through protocol-enforced adjudication.

3 Theoretical Framework: Verdict Algebra and Ring Logic

To eliminate the Industrial Liability Gap defined in Section 2, attribution must be elevated from an inferred forensic property to a deterministic protocol invariant. This requires a formal model in which authority, decision, admissibility, and conflict resolution are explicitly defined and cryptographically enforced.

We introduce a theoretical framework consisting of two complementary components:

- **Verdict Algebra**, which formalizes deterministic decision resolution, and
- **Ring Logic**, which formalizes hierarchical authority precedence and liability transfer.

Together, these constructs transform attribution into a deterministic and cryptographically verifiable function.

3.1 Authority Model

We define an authority as a cryptographically identifiable entity capable of issuing signed decisions. Formally, an authority is defined as:

$$u = (id, \sigma_{pub}, \tau)$$

where:

- id is a globally unique identifier,
- σ_{pub} is the authority's public verification key,
- τ is the authority's validity interval.

The authority set is defined as:

$$\mathcal{U} = \{u_1, u_2, \dots, u_n\}$$

Authorities include human operators, deterministic policies, and governance entities. Each authority is capable of producing signed adjudication artifacts.

3.2 Physics as an Admissibility Constraint Domain

Physical reality imposes non-negotiable constraints on admissible state transitions. These constraints do not originate from authorities and cannot be overridden by cryptographic signatures.

We define the admissibility constraint domain:

$$\Phi \subseteq \mathcal{X} \times \mathcal{A}$$

where $(x, a) \in \Phi$ represents physically admissible state-action pairs. Any action violating physical constraints is inadmissible:

$$(x, a) \notin \Phi \Rightarrow a \text{ cannot be executed}$$

Physics is therefore not modeled as an authority, but as a constraint on admissible verdicts.

This establishes a non-overridable boundary condition for adjudication.

3.3 Verdict Definition

A Verdict is a cryptographically signed adjudication artifact binding authority, system state, and decision outcome.

Formally, a verdict is defined as:

$$v = (u, x, a, t, \sigma)$$

where:

- $u \in \mathcal{U}$ is the issuing authority,
- $x \in \mathcal{X}$ is the system state,
- $a \in \mathcal{A}$ is the adjudicated action,
- t is the timestamp,
- σ is the cryptographic signature of the authority.

The verdict space is defined as:

$$\mathcal{V} = \{v \mid v \text{ is a valid verdict under the protocol}\}$$

Verdicts are immutable, non-repudiable, and independently verifiable.

In protocol implementation, this is encoded as a compact vector $v = (\delta, \alpha, \zeta, \tau, \sigma)$, where $\delta \in \{0, 1\}$ encodes the authorization outcome derived from a , $\alpha := \rho(u)$ encodes the authority precedence, σ is the cryptographic signature over $\delta, \alpha, \zeta, \tau$, and τ corresponds to t . We distinguish between an abstract verdict representation and its protocol-level encoding.

3.4 Ring Logic: Authority Precedence Ordering

Authorities are organized according to a precedence hierarchy called **Ring Logic**, which defines deterministic authority ordering.

We define a precedence function:

$$\rho : \mathcal{U} \rightarrow \mathbb{N}$$

where lower values indicate higher precedence.

For authorities $u_i, u_j \in \mathcal{U}$:

$$\rho(u_i) < \rho(u_j)$$

implies that u_i has higher precedence.

This precedence ordering defines a total order over authorities.

A typical precedence hierarchy may include:

- Ring 1: Governance and regulatory authorities
- Ring 2: Human operators
- Ring 3: Deterministic automation policies
- Ring 4: Advisory computational agents

Physical constraints defined by Φ operate outside this hierarchy as admissibility conditions.

3.5 Verdict Algebra

Multiple authorities may produce competing verdicts for a given system state.

We define the set of candidate verdicts:

$$\mathcal{V}_x = \{v_1, v_2, \dots, v_n\}$$

We define the set of admissible verdicts:

$$\mathcal{V}_{valid} = \{v \in \mathcal{V}_x \mid (x, a_v) \in \Phi\}$$

The final verdict is determined by the resolution function:

$$v_{final} = \arg \min_{v \in \mathcal{V}_{valid}} \rho(u_v)$$

where u_v is the authority associated with verdict v .

This resolution function satisfies:

- determinism,
- uniqueness,
- reproducibility.

This defines the Verdict Algebra resolution operator.

Complexity: The resolution function evaluates all $v \in \mathcal{V}_{valid}$ in a single linear scan, selecting the minimum-precedence admissible verdict. Time complexity is $O(|\mathcal{V}_x|)$ where $|\mathcal{V}_x|$ is bounded by the number of registered authorities $|\mathcal{U}|$. In practice $|\mathcal{U}|$ is small (typically < 10 authorities per domain), ensuring sub-millisecond resolution consistent with Section 5.3 measurements.

3.6 Execution Preemption and Liability Transfer

Human operators may issue signed override verdicts to supersede deterministic automation policies. We define override verdict:

$$v_{override} = (u_{human}, x, a_{override}, t, \sigma_{human})$$

If admissible under Φ , the override verdict participates in the resolution process. While under standard Ring Logic a human operator (Ring 2) already holds higher precedence than automation (Ring 3), the override mechanism serves a distinct purpose: it elevates the event from a silent precedence resolution to an explicit, ledger-anchored liability transfer. An override transaction triggers a Confession event on the ledger, shifting the system into a Dirty State and formally transferring accountability to the human authority. Without this explicit mechanism, the attribution artifact would exist only implicitly. If:

$$\rho(u_{human}) < \rho(u_{policy})$$

the override verdict participates in resolution; the override transaction additionally anchors an explicit liability transfer record to the ledger, yielding:

$$v_{final} = v_{override}$$

Unlike traditional system overrides, this process produces explicit, cryptographically verifiable attribution.

3.7 Cognitive Verdict Model

Probabilistic computational agents, such as machine learning models, cannot produce final executable verdicts.

We define cognitive authorities:

$$\mathcal{U}_C \subset \mathcal{U}$$

Cognitive verdicts:

$$v_c \in \mathcal{V}_C$$

cannot become final verdicts unless endorsed by deterministic authorities.

This ensures that probabilistic inference cannot directly produce executable actions without deterministic attribution.

3.8 Liability Conservation Law

We define attribution as a function of the final verdict:

$$\text{Attr}(e) = u_{v_{final}}$$

Because:

- verdicts are cryptographically signed,
- admissibility is deterministic,
- and resolution is deterministic,

attribution becomes deterministic and reproducible.

This establishes the **Liability Conservation Law**:

$$\forall e \in \mathcal{E}, \exists! u \in \mathcal{U} \text{ such that } \text{Attr}(e) = u$$

Liability cannot be destroyed, only transferred through verdict substitution.

3.9 Framework Guarantees

This framework guarantees:

Deterministic Attribution

Authority attribution is a deterministic function of protocol artifacts.

Cryptographic Non-repudiation

Authorities cannot deny issued verdicts.

Override Accountability

All override actions produce explicit attribution artifacts.

Physical Safety Enforcement

Actions violating physical constraints cannot be authorized.

3.10 Transition to Protocol Implementation

This theoretical framework establishes the formal basis for deterministic liability attribution.

In the following section, we present the Sopcos Protocol architecture, which implements this framework as a distributed system.

4 The Sopcos Protocol Architecture

To implement the deterministic adjudication framework defined in Section 3, the Sopcos Protocol adopts a bicameral architecture that strictly separates deterministic authority resolution from probabilistic advisory computation.

This architectural separation ensures that safety-critical adjudication remains deterministic, reproducible, and attributable, while probabilistic cognitive processes remain advisory and cannot directly produce executable authority.

Formally, the Sopcos system is defined as:

$$\mathcal{S}_{Sopcos} = (\mathcal{L}, \mathcal{A}, \mathcal{J})$$

where:

- $\mathcal{L}$ is the immutable identity and policy ledger (anchor domain),
- $\mathcal{A}$ is the resolution and cognitive gateway,
- $\mathcal{J}$ is the deterministic adjudication runtime.

These components collectively implement the Verdict Algebra and Ring Logic framework defined in Section 3.

4.1 Bicameral Design: Deterministic and Cognitive Domains

The Sopcos protocol separates adjudication into two domains:

- the **Deterministic Domain** J_{det} , and
- the **Cognitive Domain** J_{cog} .

Formally:

$$\mathcal{J}_{det} : (\mathcal{X}, \mathcal{P}, \mathcal{U}) \rightarrow \mathcal{V}$$

$$\mathcal{J}_{cog} : (\mathcal{X}) \rightarrow \mathcal{V}_{candidate}$$

The deterministic domain produces executable verdicts:

$$v_{final} \in \mathcal{V}$$

The cognitive domain produces candidate verdicts:

$$v_c \in \mathcal{V}_{candidate}$$

Candidate verdicts cannot become final verdicts unless validated by the deterministic adjudication runtime.

This enforces the Cognitive Verdict constraint defined in Section 3.

4.2 Synapse: The Deterministic Adjudication Runtime

The deterministic adjudication runtime, referred to as Synapse, implements the adjudication function defined in Section 3.

Formally:

$$\mathcal{J}_{det}(x, \mathcal{P}_x, \mathcal{U}_x) \rightarrow \mathcal{V}_x$$

Synapse executes policy artifacts within a hermetic execution environment.

Execution environment properties:

- deterministic execution semantics,
- no external state mutation,
- no network dependency during adjudication,
- architecture-independent execution behavior.

This ensures:

$$\forall x, \mathcal{J}_{det}(x) = v_{final}$$

is deterministic and reproducible.

Synapse performs adjudication only and does not execute physical actions.

This establishes separation between:

Decision domain:

$$v_{final}$$

Execution domain:

$$e = (x, a, x')$$

Physical execution remains external to adjudication.

4.3 Fail-Closed Default Deny Semantics

The protocol enforces Fail-Closed safety semantics.

Let:

$$\mathcal{V}_{valid} = \{v \in \mathcal{V}_x \mid (x, a_v) \in \Phi\}$$

If:

$$\mathcal{V}_{valid} = \emptyset$$

then:

$$v_{final} = v_{deny}^{default}$$

where:

$$v_{deny}^{default} = (u_{safety}, x, \emptyset, t, \sigma_{safety})$$

and u_{safety} is the protocol-defined safety authority.

This ensures that absence of valid authorization results in deterministic denial.

4.4 Axon: The Cognitive Resolution Gateway

The resolution gateway, referred to as Axon, implements artifact retrieval and candidate verdict generation.

Formally: Artifact resolution:

$$\mathcal{A}_{resolve}(x) \rightarrow (\mathcal{P}_x, \mathcal{U}_x)$$

Candidate verdict generation:

$$\mathcal{J}_{cog}(x) \rightarrow \mathcal{V}_{candidate}$$

Candidate verdicts:

$$v_c \in \mathcal{V}_{candidate}$$

Candidate verdicts are advisory and cannot produce executable authority directly.
Candidate verdicts must be validated through deterministic adjudication:

$$v_{final} = \arg \min_{v \in \mathcal{V}_{valid}} \rho(authority(v))$$

Axon cannot modify adjudication results.
This enforces deterministic authority precedence.

4.5 Verdict Algebra Implementation

Synapse implements the Verdict Algebra resolution function defined in Section 3.

Verdict vector:

$$v = (\delta, \alpha, \zeta, \tau, \sigma)$$

where $(\delta \in \{0, 1\})$ represents denial or authorization, $(\alpha := \rho(u))$ encodes authority precedence, (ζ) represents the Jurisdictional Scope (Target Domain), (τ) is the timestamp, and (σ) is the cryptographic signature over $(\delta, \alpha, \zeta, \tau)$ binding the verdict to its issuing authority. The compact wire encoding may transmit σ as a detached signature; however, the logical verdict tuple includes σ as a first-class component to maintain consistency with the abstract definition in Section 3.3.

Verdict resolution:

$$v_{final} = \arg \min_{v \in \mathcal{V}_{valid}} \rho(u_v)$$

Deny absorption property:

$$DENY \oplus ALLOW = DENY$$

ensures safety monotonicity.

4.6 The Core Ledger: Immutable Attribution Anchor

The ledger L serves as the immutable attribution anchor.

The ledger is not an authority:

$$\mathcal{L} \notin \mathcal{U}$$

The ledger records cryptographic hashes of adjudication artifacts:

$$h_v = H(v_{final})$$

This produces immutable attribution evidence.

The ledger maintains:

- authority identity records,
- policy artifact hashes,
- verdict artifact hashes.

The ledger does not execute adjudication.

This satisfies the Thin Ledger principle.

4.7 Verdict Lifecycle

The verdict lifecycle consists of five steps:

Step 1: State observation

$$x \in \mathcal{X}$$

Step 2: Artifact resolution

$$(\mathcal{P}_x, \mathcal{U}_x) = \mathcal{A}_{resolve}(x)$$

Step 3: Candidate verdict generation

$$\mathcal{V}_{candidate} = \mathcal{J}_{cog}(x)$$

Step 4: Deterministic adjudication

$$v_{final} = \mathcal{J}_{det}(x, \mathcal{P}_x, \mathcal{U}_x)$$

Step 5: Ledger anchoring

$$h_v = H(v_{final}) \rightarrow \mathcal{L}$$

4.8 Deterministic Attribution

Attribution is defined as:

$$\text{Attr}(e) = \text{authority}(v_{final})$$

Because:

- adjudication is deterministic,
- verdicts are signed,
- verdict hashes are immutable,

attribution becomes deterministic and independently verifiable.

4.9 Architecture Guarantees

The Sopcos architecture guarantees:

Deterministic adjudication:

$$\forall x, \mathcal{J}_{det}(x) = v_{final}$$

Cryptographic attribution:

$$\text{Attr}(e) = \text{authority}(v_{final})$$

Independent verifiability:

$$\text{Verify}(H(v_{final}))$$

Fail-closed safety enforcement:

$$\mathcal{V}_{valid} = \emptyset \Rightarrow DENY$$

Liability conservation:

$$\forall e \in \mathcal{E}, \exists! u \in \mathcal{U}$$

4.10 Architectural Completeness

The Sopcos architecture provides a concrete implementation of the Verdict Algebra and Ring Logic framework.

This transforms liability attribution from a forensic reconstruction problem into a deterministic protocol computation.

In the following section, we demonstrate the protocol's operation through a canonical industrial incident scenario.

5 Case Study and Validation: Canonical Industrial Incident Scenario (CIIS)

To validate the Sopcos Protocol Architecture and the Verdict Algebra defined in Sections 3 and 4, we present a two-part evaluation: (1) a formal analytical trace (Section 5.2) that demonstrates deterministic attribution as a logical consequence of the protocol definitions, and (2) empirical validation through a reference implementation (Section 5.3) that measures real-world execution performance. We analyze the Canonical Industrial Incident Scenario (CIIS) involving a high-pressure industrial boiler.

This scenario demonstrates the transition from deterministic protocol enforcement to human override, followed by cryptographically verifiable forensic attribution. The objective is to show that attribution emerges as a deterministic protocol property rather than an interpretive forensic inference.

5.1 System Model and Adjudication Parameters

We consider a boiler unit governed by the Sopcos adjudication protocol. System state is defined as:

$$x = (P, T, H, M)$$

where:

- P is internal pressure,
- T is temperature,
- H is vibration (health/wear),
- M is motor RPM.

Maximum safe operating pressure:

$$P_{\text{safe}} = 100 \text{ bar}$$

Physical admissibility constraint domain:

$$\Phi = \{(x, a) \mid P < P_{\text{critical}}\}, \quad P_{\text{critical}} = 150 \text{ bar}$$

Authority set:

$$\mathcal{U} = \{u_{human}, u_{policy}\}$$

Authority precedence (lower ρ = higher adjudicative power):

$$\rho(u_{human}) < \rho(u_{policy})$$

Note: The protocol-defined safety authority u_{safety} used in Section 4.3 for fail-closed default-deny verdicts is a protocol-internal entity, distinct from physical constraints. It is formally defined as part of the protocol runtime (Synapse), not as an external authority in $\mathcal{U}$. The fail-closed verdict $v_{deny}^{default} = (u_{safety}, x, \emptyset, t, \sigma_{safety})$ is issued by Synapse itself when $\mathcal{V}_{valid} = \emptyset$.

Protocol components:

- **Synapse**: deterministic adjudication runtime ($\mathcal{J}_{det}$)
- **Axon**: cognitive gateway and artifact resolver
- **Ledger** ($\mathcal{L}$): immutable verdict anchor

Independent observer: auditor with read access to ledger and policy artifacts.

5.2 Formal Analytical Trace

5.2.1 Phase I: Deterministic Policy Evaluation (Machine Innocence)

Telemetry input:

$$x_1 = (P = 105, T = 480, H = 0.13, M = 1527)$$

Safety policy deterministically identifies a critical correlation between pressure and vibration:

$$P_{safety}(x) = \begin{cases} \text{DENY} & \text{if } P > P_{safe} \wedge H > 0.10 \\ \text{ALLOW} & \text{otherwise} \end{cases}$$

Synapse evaluates the policy and produces a HALT/DENY verdict:

$$v_{policy} = (u_{policy}, x_1, \text{DENY}, t_1, \sigma_{policy})$$

Final verdict is deterministically anchored:

$$v_{final} = v_{policy}, \quad h_1 = H(v_{final}) \rightarrow \mathcal{L}$$

This establishes non-liability of the automation layer.

5.2.2 Phase II: Human Override and Liability Transfer

Under standard Ring Logic, a human operator (Ring 2) already holds higher precedence than the automation policy (Ring 3). However, a silent precedence resolution is insufficient for liability attribution: the override mechanism serves the distinct purpose of elevating the event to an explicit, ledger-anchored liability transfer. The operator issues a signed override verdict, triggering a **Confession** event on the ledger, shifting the system into a Dirty State, and formally transferring accountability to the human authority.

The operator initiates an override decision to maintain operational continuity. Override verdict:

$$v_{override} = (u_{human}, x_1, \text{ALLOW}, t_2, \sigma_{human})$$

Signature verification ensures $Verify(\sigma_{human}) = true$. Resolution using Verdict Algebra applies the precedence ordering $\rho(u_{human}) < \rho(u_{policy})$, thereby deterministically selecting the override verdict as the final adjudicated outcome:

$$v_{final} = v_{override}$$

Ledger anchoring transfers liability to the human operator:

$$h_2 = H(v_{override}) \rightarrow \mathcal{L}, \quad Attr : \mathcal{E} \rightarrow \mathcal{U}, \quad Attr(e_2) = u_{human}$$

5.2.3 Phase III: Forensic Replay and Auditor Verification

After the override, the system enters a cryptographically enforced *Dirty State*. An independent auditor retrieves $h_2 \in \mathcal{L}$ and reconstructs attribution. The auditor verifies:

$$H(v_{override}) = h_2$$

Thus:

$$Attr(e_2) = u_{human}$$

is deterministic and reproducible, attribution ambiguity is eliminated. The system is restored to a **CLEAN STATE** only after the auditor submits a cryptographically signed `StateReset` transaction to the ledger.

5.3 Empirical Validation: Reference Implementation and Performance Metrics

To empirically validate the theoretical framework, we implemented the Canonical Industrial Incident Scenario (CIIS) using the Sopcos reference stack (v1.0), observing the execution latencies and deterministic anchoring behaviors during the Grand Unification Tests (SIP-019). Sopcos Improvement Proposals (SIPs) are the protocol’s formal specification documents governing architecture, semantics, and validation procedures; SIP-019 defines the canonical end-to-end validation suite for the adjudication runtime [7].

The implementation utilizes a Go-based core ledger running a Proof-of-Authority (PoA) consensus mechanism. The Synapse adjudication runtime leverages a hermetic WebAssembly (WASM) sandbox (Wazero) to execute the “Smart Unit” policy. To guarantee deterministic compilation, the policy logic was transpiled and compiled using TinyGo, yielding an artifact hash of:

278a9ac890b498c5e9596d7a2efcd28f1477fc9016c9a21f6eb3e8fdf4863a0d.

Execution Performance: During high-frequency telemetry injection (evaluating four parameters: Pressure, Temperature, Vibration, and RPM), the Synapse runtime demonstrated strict sub-millisecond adjudication latency. Excluding the initial WASM cold start (~ 1.3 ms), the hot-path evaluations consistently executed between $221.2 \mu s$ and $580.8 \mu s$. These results confirm that deterministic adjudication can be performed within strict industrial latency constraints without compromising cryptographic accountability. These metrics validate that the protocol satisfies the ultra-low latency requirements (< 10 ms) for industrial hot-path enforcement, consistent with the response-time bounds specified in IEC 62443-3-3 for real-time control loops [2] and NIST SP 800-82 guidance for ICS communication [4].

Cryptographic Anchoring and State Transitions: The implementation successfully verified the *Execution Preemption* mechanics defined in our Ring Logic. When the operator bypassed the algorithm using the Vinci Wallet, the Axon L2 gateway successfully broadcast a `Confession` transaction — a protocol-defined ledger event (SIP-006) that records the operator’s DID, the override justification code, and the hash of the superseded policy verdict, formally transitioning the system to Dirty State. This immediately transitioned the execution environment into a Dirty State (defined as a state where deterministic policy compliance is

suspended due to an explicit override), appending the operator’s Decentralized Identifier (DID) and justification code (CLASS_S or CLASS_C) to the public ledger. Subsequent operations were structurally locked until a Level 5 Auditor evaluated the physical asset and issued a `StateReset` (OpCode `0x12`) transaction, officially closing the liability gap and restoring the Clean State.

5.4 Empirical Validation Result

The SIP-019 execution trace empirically confirms that liability attribution in the Sopcos Protocol is a deterministic function of adjudicated authority, independent of post-hoc interpretation.

Formally:

$$\forall e \in \mathcal{E}, \exists! u \in \mathcal{U} : \text{Attr}(e) = u$$

Post-hoc interpretation ambiguity is eliminated.

5.5 Reproducibility Artifact

A full execution trace of the SIP-019 validation, including telemetry, verdict transitions, and ledger anchoring, is available at: <https://youtu.be/V8Qj3fK0gQg>

6 Security Analysis and Threat Model Validation

In this section, we analyze the security properties of the Sopcos Protocol under an adversary model whose objective is not service disruption or financial fraud, but liability evasion.

Unlike traditional blockchain systems where the primary adversarial objective is double-spending, or conventional industrial control systems where the primary threat is denial of service, the primary adversarial objective in the Sopcos threat model is to induce a catastrophic physical outcome while preventing deterministic attribution.

Formally, the adversary attempts to induce a failure state:

$$S_{fail}$$

while causing the attribution function:

$$\text{Attr}(e)$$

to produce either:

$$\emptyset \quad \text{or} \quad u_{false}$$

where u_{false} is an incorrect authority.

We demonstrate that under the Sopcos protocol, such attribution evasion is computationally and algebraically impossible.

6.1 Adversary Model

We define adversary A as a computationally bounded entity capable of controlling a subset of system components.

Formally:

$$A \subseteq C$$

Adversary capabilities include:

Key compromise

Adversary may possess valid signing keys:

$$\sigma_u \text{ for some } u \in \mathcal{U}$$

including human operators or cognitive agents.

Protocol compliance

Adversary may produce syntactically valid verdicts:

$$v = (u, x, a, t, \sigma)$$

that pass signature verification.

This constitutes the Evil but Compliant adversary defined in Section 2.

Sensor manipulation

Adversary may provide arbitrary telemetry input:

$$x_{adv}$$

to the adjudication runtime.

Cognitive model manipulation

Adversary may deploy malicious or poisoned cognitive models producing candidate verdicts:

$$v_c \in \mathcal{V}_{candidate}$$

Adversary cannot break:

- digital signature unforgeability,
- cryptographic hash collision resistance,
- ledger immutability.

6.2 Authority Hijacking Resistance (Ring Logic Invariant)

Attack objective:

Adversary attempts to authorize unsafe action using compromised low-precedence authority.

Malicious verdict:

$$v_{malicious} = (ALLOW, \alpha = 3, \sigma, t)$$

Active safety constraint verdict:

$$v_{constraint} = (DENY, \alpha = 1, \sigma, t)$$

Resolution function:

$$v_{final} = \arg \min_{v \in \mathcal{V}_{valid}} \alpha_v$$

Since:

$$\alpha_{constraint} < \alpha_{malicious}$$

resolution yields:

$$v_{final} = v_{constraint}$$

Unsafe authorization is algebraically impossible.

This establishes the Ring Logic invariant:

Lower-precedence authorities cannot override higher-precedence constraints.

6.3 Oracle Manipulation and Input Provenance Binding

Adversary may provide false telemetry:

$$x_{false}$$

Adjudication produces verdict:

$$v_{final} = (u, x_{false}, a, t, \sigma)$$

Ledger anchoring:

$$h_v = H(v_{final})$$

Input hash binding:

$$h_x = H(x_{false})$$

Verdict contains cryptographic reference to input hash:

$$v_{final} \supseteq h_x$$

Forensic replay reveals:

$$Verify(v_{final}, h_x) = true$$

but physical inspection shows:

$$x_{false} \neq x_{real}$$

Attribution becomes:

$$Attr(e) = authority(v_{final})$$

This converts oracle manipulation from an attribution-obscuring attack into an attributable failure.

The protocol does not prevent false input, but ensures deterministic attribution of its origin.

This eliminates attribution ambiguity.

6.4 Cognitive Layer Manipulation Resistance

Adversary may deploy malicious cognitive model producing unsafe recommendation:

$$v_c = (ALLOW, \alpha = 3, \sigma, t)$$

Candidate verdict set:

$$\mathcal{V}_{candidate}$$

Final verdict resolution remains:

$$v_{final} = \arg \min_{v \in \mathcal{V}_{valid}} \alpha_v$$

Cognitive verdict cannot override deterministic authority.

Model provenance binding:

$$h_{model} = H(model)$$

Verdict contains model reference:

$$v_c \supseteq h_{model}$$

Forensic replay:

$$model(x) = a$$

is reproducible.

Attribution becomes traceable to model authorization authority.

This eliminates the Probabilistic Determinism Paradox.

6.5 Data Availability and Artifact Integrity Protection

Adversary may attempt to remove policy artifacts to obscure attribution.

Ledger stores immutable artifact hash:

$$h_{policy} = H(P)$$

If artifact is removed, ledger retains hash.

Forgery requires finding:

$$P' \neq P \quad \text{such that} \quad H(P') = H(P)$$

which is computationally infeasible.

Artifact integrity remains provable.

6.6 Fail-Closed Vacuum State Protection

Adversary may attempt to disrupt policy retrieval.

If valid verdict set becomes empty:

$$\mathcal{V}_{valid} = \emptyset$$

protocol enforces default safety verdict:

$$v_{final} = v_{deny}^{default}$$

This ensures absence of policy cannot produce unsafe authorization.

Fail-open state is algebraically impossible.

6.7 Authority Repudiation Resistance

Authority may attempt to deny override action.

Override verdict:

$$v_{override} = (u, x, a, t, \sigma_u)$$

Ledger anchoring:

$$h_v = H(v_{override})$$

Signature verification ensures:

$$Verify(\sigma_u) = true$$

Authority attribution remains provable.

Repudiation becomes cryptographically impossible.

6.8 Attribution Integrity Theorem

Theorem 1 (Attribution Preservation under Adversarial Conditions). *Given:*

- *deterministic adjudication function,*
- *cryptographically signed verdicts,*
- *immutable ledger anchoring,*

then attribution remains deterministic under adversarial conditions.

Formally:

$$\forall e, \exists! u \in \mathcal{U} \quad \text{such that} \quad Attr(e) = authority(v_{final})$$

Adversary cannot produce null attribution:

$$Attr(e) \neq \emptyset$$

This establishes attribution determinism as a protocol invariant.

6.9 Security Guarantees Summary

Under the defined threat model, Sopcos guarantees:

- deterministic attribution
- cryptographic non-repudiation
- authority precedence enforcement
- cognitive isolation
- artifact integrity preservation
- fail-closed safety behavior

These guarantees eliminate liability evasion as an admissible adversarial outcome.

7 Related Work and Comparative Analysis

This section situates the Sopcos Protocol within the broader landscape of industrial control systems, blockchain architectures, secure logging systems, trusted execution environments, and AI safety frameworks.

Unlike existing systems, Sopcos does not merely secure execution or ensure data integrity; it establishes deterministic and cryptographically verifiable liability attribution as a protocol-level property.

7.1 Industrial Control Systems and SCADA Architectures

Traditional Industrial Control Systems (ICS) and SCADA platforms are designed to monitor and control physical processes through centralized supervisory logic.

These systems maintain execution logs recording observed state transitions:

$$(x, a, x')$$

However, these logs do not cryptographically bind execution to originating authority.

As discussed in Section 2, this creates Epistemic Opacity: multiple authorities remain consistent with observed execution.

Existing SCADA security enhancements focus primarily on:

- network segmentation,
- authentication mechanisms,
- anomaly detection,
- and intrusion prevention.

These measures improve operational security but do not provide deterministic attribution.

In contrast, Sopcos introduces cryptographically signed adjudication artifacts:

$$v = (u, x, a, t, \sigma)$$

which bind authority to decision at protocol level.

This transforms attribution from an interpretive process into a deterministic computation. Furthermore, IEC 62443 provides defense-in-depth through network segmentation and access control. Its security model is access-centric: once an authenticated entity is inside a zone, the standard does not evaluate the semantic validity of commands issued by that entity (Conduit/Zone model, IEC 62443-3-3) [2]. Sopcos is strictly complementary: where IEC 62443 controls ingress, Sopcos adjudicates intent. Together they form a layered architecture — access is enforced *ex ante* by IEC 62443; liability is deterministically resolved *ex post* by Sopcos.

7.2 Perimeter Security vs. Liability Attribution (IEC 62443 / ISA-99)

The ISA/IEC 62443 series of standards represents a widely adopted international framework for securing Industrial Automation and Control Systems (IACS). It is built upon the foundational concepts of *Zones and Conduits*, enabling defense-in-depth through network segmentation, controlled communication paths, and robust access control mechanisms.

The primary objective of IEC 62443 is to ensure perimeter and transport security by preventing unauthorized actors from accessing industrial networks or injecting malicious commands. However, its security model is fundamentally access-centric. Once an entity (human operator or autonomous controller) is authenticated and authorized within a given zone, the standard does not inherently evaluate the semantic correctness or systemic consequences of operational commands issued by that entity.

This limitation gives rise to what we define as the *Industrial Liability Gap*: the absence of a deterministic and verifiable mechanism to attribute responsibility for system-level outcomes to specific authorized actors. In such cases, a fully authenticated and authorized entity may issue a catastrophic yet syntactically valid command. While IEC 62443 ensures that the command originates from a trusted source, it does not provide a mechanism to cryptographically bind the executed action, its originating authority, and the system state to the resulting outcome.

The Sopcos Protocol is designed to be strictly complementary to IEC 62443. While IEC 62443 secures the *network perimeter* against unauthorized access, Sopcos secures the *decision space*, defined as the set of admissible actions evaluated under deterministic policy and authority constraints. In a unified architecture, IEC 62443 ensures that only permitted actors can access the control network, whereas Sopcos (via Verdict Algebra and the L1 Ledger) ensures that every action taken by those actors is deterministically evaluated, constrained by physical *Pre-Law* (i.e., non-negotiable safety and physical constraints encoded as policy), and immutably anchored for forensic liability attribution.

Conceptually, IEC 62443 enforces access control at the system boundary, whereas Sopcos provides deterministic adjudication and immutable recording of actions within that boundary. Together, they form a layered architecture in which security and accountability are jointly enforced: access is controlled *ex ante*, and liability is deterministically resolved *ex post*.

7.3 Regulatory Context and Deterministic Accountability (EU Machinery Regulation 2023/1230)

The EU Machinery Regulation 2023/1230 establishes a comprehensive legal framework governing the safety, integrity, and reliability of machinery systems, with particular emphasis on software-enabled and semi-autonomous operation. While the regulation mandates strict requirements for system safety, resilience against corruption, and human oversight, it does not define a deterministic technical mechanism for attributing responsibility across complex human-machine interaction layers.

In particular, Annex III (Essential Health and Safety Requirements) introduces explicit obligations regarding protection against corruption (Section 1.1.9) and the safety and reliability of control systems (Section 1.2.1). These provisions require that safety-related software and hardware be protected against unauthorized modification, accidental corruption, and foreseeable malicious interference. Furthermore,

systems must retain verifiable records of interventions and decision-making processes to enable post-event auditability by competent authorities.

However, as discussed in Section 2, conventional industrial control architectures typically satisfy these requirements through localized and mutable logging mechanisms, which suffer from epistemic opacity and lack cryptographic non-repudiation. As a result, while compliance may be nominally achieved, deterministic responsibility attribution remains unresolved.

This gap reflects a broader distinction between regulatory intent and technical enforceability. The regulation defines *what* must be ensured—integrity, traceability, and accountability—but does not prescribe *how* these properties should be enforced at the level of decision execution.

Sopcos addresses this gap by providing a deterministic and cryptographic mechanism that can be used to satisfy these requirements through protocol-level invariants:

- **Protection against Corruption (EHSR 1.1.9):** Ensured via cryptographically signed and hash-anchored execution artifacts (SIP-009, SIP-014). Safety policies and AI models execute within a hermetic WebAssembly (WASM) environment, where any tampering results in a hash mismatch and deterministic *fail-closed* rejection.
- **Evidence of Intervention (EHSR 1.1.9 & 1.2.1):** Human overrides are structured as cryptographically signed Execution Preemptions (SIP-006) rather than simple system alerts. This explicitly shifts the system into a verifiable *Dirty State*, anchoring an immutable liability record to the operator’s Decentralized Identifier (DID) on the Core Ledger.
- **Decision-Making Transparency (EHSR 1.2.1):** The Cognitive Verdict mechanism (SIP-015) ensures that probabilistic AI recommendations do not directly actuate machinery. Instead, the logic and exact model weights are cryptographically bound to candidate verdicts, enabling deterministic forensic replayability.

Importantly, these mechanisms do not merely enforce traditional cybersecurity properties such as integrity and access control; they extend them to deterministic responsibility attribution across human–machine interaction layers.

Consequently, Sopcos extends conventional compliance approaches by providing a cryptographically verifiable execution layer in which safety constraints, authority, and system state are jointly evaluated and immutably recorded. In this sense, while EU 2023/1230 defines the normative expectations for safe and trustworthy machinery, Sopcos provides a concrete protocol-level mechanism through which these expectations can be systematically enforced and audited in practice.

7.4 Blockchain Systems and Distributed Ledger Technologies

Blockchain systems such as Bitcoin and Ethereum provide decentralized consensus and immutable transaction records.

These systems ensure integrity of transaction ordering and prevent double-spending.

However, blockchain systems operate on a transaction-centric model:

$$Tx = (sender, receiver, value, signature)$$

They do not model authority precedence, admissibility constraints, or deterministic adjudication.

Smart contract platforms provide programmable execution environments but lack explicit authority hierarchy and deterministic liability attribution mechanisms.

Smart contracts execute code deterministically, but attribution remains limited to transaction signers rather than adjudicated authority.

Sopcos differs fundamentally in its Thin Ledger architecture:

$$\mathcal{L} \notin \mathcal{U}$$

The ledger anchors adjudication artifacts rather than executing business logic.
Authority attribution is derived from adjudicated verdicts rather than transaction origin.
This enables protocol-level liability attribution rather than transaction-level provenance.

7.5 Secure Logging and Tamper-Evident Logging Systems

Secure logging systems provide integrity guarantees for recorded events through hash chaining and digital signatures.

Examples include forward-secure logging systems such as the scheme of Schneier and Kelsey [8], Certificate Transparency logs [9], and the Sigstore artifact transparency framework [10]. These systems guarantee tamper-evidence of recorded events but do not resolve attribution ambiguity: multiple authorities remain consistent with any given execution record, precisely because adjudicated decisions are not anchored prior to execution.

These systems ensure that recorded events cannot be modified without detection.
However, secure logging systems record execution events:

$$(x, a, x')$$

rather than adjudicated decisions.
As a result, they preserve execution integrity but do not resolve attribution ambiguity.
Multiple authorities may remain consistent with recorded execution.
Sopcos introduces cryptographically signed adjudication artifacts prior to execution:

$$v_{final}$$

which uniquely determine attribution.
This eliminates attribution ambiguity at the protocol level.

7.6 Trusted Execution Environments

Trusted Execution Environments (TEEs), including Intel SGX [11], ARM TrustZone [12], and RISC-V Keystone [13], provide hardware-enforced isolation for security-sensitive computation. Like Synapse’s WASM sandbox, TEEs aim to ensure that policy execution cannot be tampered with by the host operating system. However, TEEs rely on hardware vendor attestation, introducing a trust dependency on silicon manufacturers and firmware supply chains. Sopcos’s WASM-based approach provides architecture-independent determinism without hardware trust assumptions, at the cost of weaker isolation guarantees than hardware enclaves. TEE integration remains a promising future direction, potentially providing hardware-backed key storage and enclave-attested adjudication.

7.7 Comparative Summary Table

The preceding analysis highlights that existing systems address execution integrity, transaction finality, or operational optimization, but do not provide deterministic and cryptographically verifiable liability attribution.

Traditional SCADA systems ensure operational continuity but rely on mutable audit logs and implicit trust assumptions. General-purpose blockchain platforms ensure transaction immutability but lack awareness

Feature	SCADA / ICS	Smart Contracts	AI / AIOps	Sopcos Protocol
Primary Goal	Execution & Control	Transaction Finality	Optimization	Liability Attribution
Trust Model	Centralized / Implicit	Decentralized / Code-based	Probabilistic	Cryptographic
Conflict Resolution	Last-write-wins	Transaction Order	Model Weighting	Verdict Algebra
Latency	<10ms	>2s	Variable	<10ms
Physics Awareness	None	None	Approximate	Ring 0 Constraint
Privacy	Low	Low	N/A	High
Attribution Mechanism	Mutable logs / inferred	Transaction signer	N/A (probabilistic)	Deterministic verdict algebra

Table 1: Comparative Analysis

* Latency figures for SCADA/ICS and Sopcos represent hot-path execution bounds. SCADA figure reflects typical PLC scan-cycle latency in process automation. Sopcos figure is empirically validated in Section 5.3 (221–581 μs , excluding WASM cold start).
Smart contract figure reflects median finality time on public blockchain networks.

of physical admissibility constraints and do not resolve authority precedence. AI-driven operational systems provide optimization capabilities but introduce probabilistic decision-making and attribution ambiguity.

In contrast, the Sopcos Protocol establishes liability attribution as a protocol-level invariant. This is achieved through the combination of Verdict Algebra, Ring Logic authority ordering, and Thin Ledger anchoring of adjudication artifacts.

Table 1 summarizes the architectural and functional distinctions between Sopcos and prior paradigms.

7.8 AI Safety Frameworks and Model Governance Systems

Recent work in AI safety has focused on model alignment, interpretability, and governance mechanisms to mitigate unsafe behavior.

These approaches include:

- model auditing,
- interpretability tools,
- and policy-based AI governance.

However, these systems do not establish deterministic attribution of AI-influenced actions.

AI decisions remain probabilistic and attribution often remains ambiguous.

Sopcos introduces the Cognitive Verdict Model:

$$v_c \in \mathcal{V}_{candidate}$$

AI systems produce candidate verdicts that require deterministic endorsement before execution.

This ensures that attribution remains deterministic even in AI-assisted decision systems.

This eliminates the Liability Vacuum associated with autonomous decision-making systems.

7.9 Comparative Summary

The fundamental distinction between Sopcos and existing systems lies in its treatment of attribution as a protocol invariant rather than a forensic inference.

Existing systems ensure:

- execution integrity,
- transaction integrity,

- or computation integrity.

Sopcos uniquely ensures:

$$Attr(e) = authority(v_{final})$$

as a deterministic protocol property.

This transforms liability attribution from an interpretive process into a mathematically verifiable function.

7.10 Contribution Relative to Existing Systems

The Sopcos Protocol introduces three novel contributions relative to prior work:

1. A formal Verdict Algebra enabling deterministic authority resolution.
2. A hierarchical authority model (Ring Logic) enforcing precedence constraints.
3. A Thin Ledger architecture anchoring adjudication artifacts rather than transactions.

These contributions collectively establish protocol-based liability attribution.

8 Discussion and Implications

The architectural validation presented in Section 5 and the comparative analysis in Section 7 demonstrate that the Sopcos Protocol establishes deterministic liability attribution as a protocol-level property. This section examines the broader implications of this capability across safety engineering, industrial economics, artificial intelligence governance, and legal attribution frameworks.

8.1 Mitigation of Moral Hazard through Deterministic Attribution

Traditional industrial safety standards, including IEC 61508 and ISO 26262, focus on system reliability and compliance. These frameworks assume that safety constraints, once implemented, will be followed in practice. However, they do not address the behavioral phenomenon of moral hazard, where human operators may override safety mechanisms when attribution risk is ambiguous or unenforceable.

Under conventional SCADA architectures, manual overrides often leave incomplete or mutable records, allowing attribution ambiguity.

In Sopcos, every override produces a cryptographically signed verdict:

$$v_{override} = (u_{human}, x, a, t, \sigma_{human})$$

which is immutably anchored to the ledger:

$$h_v = H(v_{override})$$

This produces deterministic attribution:

$$Attr(e) = u_{human}$$

As a result, override actions become explicitly attributable events.

This alters operator decision dynamics by introducing guaranteed attribution, reducing the incentive to bypass safety constraints without explicit acceptance of responsibility.

This establishes attribution as an architectural safety mechanism rather than a post-hoc forensic outcome.

8.2 Economic Implications: Usage-Based Industrial Insurance

Deterministic attribution enables new economic models for industrial risk assessment and insurance.

The protocol explicitly defines operational compliance state as:

$$State \in \{Clean, Dirty\}$$

where Dirty State corresponds to periods during which deterministic safety policies are overridden by higher-precedence authority.

The ledger provides immutable records of Dirty State intervals:

$$\Delta t_{dirty}$$

This enables insurers to compute objective risk metrics directly from protocol evidence.

Insurance pricing may be derived from measurable protocol-level risk exposure rather than retrospective audits.

This enables usage-based industrial insurance models where premiums reflect provable operational behavior rather than inferred risk — analogous to telematics-based automotive insurance [14], but grounded in cryptographically verifiable protocol evidence rather than self-reported telemetry. Unlike parametric IoT insurance models [15] that rely on trusted third-party data feeds, Sopcos provides a tamper-evident attribution record that can serve directly as actuarial evidence.

This aligns economic incentives with safe operation.

8.3 Safe Integration of Artificial Intelligence in Safety-Critical Systems

Artificial intelligence introduces probabilistic decision-making into deterministic safety environments.

This creates a fundamental conflict between optimization capability and accountability requirements.

The Sopcos protocol resolves this conflict through architectural authority separation.

Cognitive agents produce candidate verdicts:

$$v_c \in \mathcal{V}_{candidate}$$

which cannot become executable verdicts without deterministic adjudication.

Final authority resolution remains governed by Verdict Algebra:

$$v_{final} = \arg \min_{v \in \mathcal{V}_{valid}} \rho(authority(v))$$

This ensures that probabilistic cognitive agents cannot directly produce executable authority.

Additionally, cognitive verdicts include cryptographic references to model identity:

$$h_{model} = H(model)$$

This enables deterministic forensic replay of AI-influenced decisions.

This establishes a framework for accountable deployment of AI in safety-critical systems.

8.4 Resolution of the Decision Provenance Problem and the Oracle Boundary

The Sopcos protocol establishes deterministic provenance of decisions, but does not intrinsically guarantee the correctness of physical measurements.

Adversarial telemetry input:

$$x_{false}$$

produces adjudicated verdict:

$$v_{final} = (u, x_{false}, a, t, \sigma)$$

Ledger anchoring ensures deterministic attribution:

$$Attr(e) = authority(v_{final})$$

This ensures attribution of decision authority even when physical inputs are incorrect.

The protocol does not eliminate the possibility of false inputs, but ensures that false inputs remain attributable.

This converts attribution ambiguity into attributable fault.

This represents the maximal achievable guarantee at the digital-physical boundary.

8.5 Implications for Asset Governance and Autonomous Resource Control

The integration of adjudicated authority with industrial asset identity enables deterministic governance of industrial resources.

Let asset state be represented as:

$$A = (asset_id, state, authority)$$

Operational admissibility may be defined as:

$$Admissible(x, A)$$

Execution becomes conditional on asset compliance state.

This enables deterministic enforcement of operational constraints derived from regulatory, contractual, or physical requirements.

This extends protocol-based attribution to protocol-based resource governance.

8.6 Emergence of Attribution-Native Cyber-Physical Systems

Traditional cyber-physical systems are execution-centric.

Sopcos introduces attribution-centric system architecture.

Execution becomes a consequence of adjudicated authority rather than implicit control logic.

This establishes attribution as a first-class protocol property.

This architectural model may enable:

- deterministic forensic reconstruction,
- automated regulatory compliance verification,
- protocol-level enforcement of operational authority constraints.

This represents a shift from trust-based operational systems to attribution-native systems.

9 Limitations and Future Work

While the Sopcos Protocol establishes deterministic liability attribution at the adjudication layer, its guarantees operate within defined epistemic, architectural, and physical boundaries. This section defines the scope of these guarantees and outlines future research directions.

9.1 The Epistemic Boundary: The Oracle Problem

The Sopcos protocol guarantees deterministic attribution of decisions, but does not intrinsically guarantee the correctness of physical measurements.

Let:

$$x_{real}$$

represent physical reality, and

$$x_{reported}$$

represent telemetry input provided to the adjudication runtime.

The protocol computes adjudicated verdict:

$$v_{final} = \mathcal{J}_{det}(x_{reported})$$

and anchors attribution:

$$Attr(e) = authority(v_{final})$$

If:

$$x_{reported} \neq x_{real}$$

the protocol cannot independently detect the discrepancy.

This limitation reflects a fundamental epistemic boundary common to all cyber-physical systems.

However, the protocol guarantees that the source of reported telemetry remains cryptographically attributable.

This converts attribution ambiguity into attributable fault.

Future work may explore integration with trusted sensor hardware, multi-source telemetry consensus, and cryptographic sensor attestation.

9.2 Latency Asymmetry Between Adjudication and Ledger Anchoring

The Sopcos architecture separates deterministic adjudication from immutable anchoring.

Adjudication occurs in real time:

$$t_{adjudication} < 10ms$$

while ledger anchoring occurs asynchronously:

$$t_{anchor} > t_{adjudication}$$

This introduces a temporal gap between decision execution and proof anchoring.

During network disruption, adjudication artifacts may remain locally buffered until connectivity is restored.

If catastrophic physical destruction occurs before anchoring, some adjudication artifacts may be lost.

This limitation reflects the physical dependency of digital storage infrastructure.

Future work includes redundant anchoring mechanisms, distributed artifact replication, and multi-ledger anchoring strategies.

9.3 Dependency on Secure Key Management

The protocol assumes that private signing keys remain under the control of designated authorities.

If private key:

$$\sigma_u$$

is compromised, adversary may produce valid verdicts attributable to authority u .

Attribution remains deterministic but authority authenticity becomes compromised.

Future work includes integration with hardware-backed key storage, secure enclaves, distributed key custody, and formal key revocation mechanisms.

9.4 Integration with Legacy Industrial Infrastructure

Sopcos operates as an overlay adjudication protocol rather than a replacement for existing industrial control systems.

Legacy infrastructure may lack native support for cryptographic signing or deterministic adjudication interfaces.

Deployment therefore requires gateway infrastructure capable of producing and validating adjudication artifacts.

Future work includes development of lightweight adjudication clients suitable for resource-constrained industrial devices and deeper integration into sensor and controller layers.

9.5 Extension Toward Asset-Native Attribution Systems

The current protocol version establishes deterministic attribution of decisions.

Future work extends attribution to asset lifecycle and resource governance.

Industrial assets may be represented as cryptographically identifiable entities:

$$Asset = (asset_id, authority, state)$$

This enables deterministic attribution of asset state transitions, ownership, and operational authority.

This extends attribution guarantees beyond decisions to industrial asset governance.

9.6 Predictive Adjudication and Temporal Liability

Current adjudication operates on present system state:

$$v_{final} = \mathcal{I}_{det}(x_{present})$$

Future research may extend adjudication to predicted system state:

$$x_{future}$$

Cognitive agents may produce predictive verdict candidates:

$$v_{future} \in \mathcal{V}_{candidate}$$

Deterministic anchoring of predictive verdicts may enable attribution of failure to prior ignored warnings.

This introduces the concept of temporal attribution, where liability may be attributed to failure to act on prior attributable warnings.

This represents a new research direction at the intersection of predictive systems and liability attribution.

9.7 Scalability and Distributed Deployment Considerations

While adjudication complexity remains bounded $O(|\mathcal{V}_x|)$ — linear in the number of candidate verdicts for a given system state — global deployment across industrial infrastructure requires scalable anchoring and artifact management. Here $|\mathcal{V}_x|$ denotes the cardinality of the candidate verdict set, which is bounded in practice by the number of registered authorities.

Future work includes hierarchical anchoring, artifact aggregation, and distributed ledger architectures optimized for attribution anchoring.

9.8 Summary of Protocol Boundaries

The Sopcos Protocol guarantees:

- deterministic attribution of adjudicated decisions,
- cryptographically verifiable authority provenance,
- immutable anchoring of adjudication artifacts.

The protocol does not guarantee:

- correctness of physical measurements,
- protection against physical destruction of infrastructure,
- or immunity to compromised authority keys.

These limitations reflect fundamental constraints at the cyber-physical boundary.

Future work aims to extend attribution guarantees deeper into sensing, asset management, and predictive decision domains.

10 Conclusion

As industrial cyber-physical systems transition from passive monitoring to semi-autonomous operation, the inability to deterministically attribute operational decisions creates an untenable Industrial Liability Gap — one that traditional SCADA architectures and blockchain platforms fail to resolve. This paper makes five concrete contributions toward closing this gap. First, we formalized the Industrial Liability Gap as an architectural property — not merely an operational shortcoming — of existing ICS and SCADA systems (Section 2). Second, we introduced protocol-based liability attribution as a deterministic alternative to post-hoc forensic reconstruction. Third, we formalized the Verdict Algebra and Ring Logic, providing a mathematically rigorous framework for conflict resolution across heterogeneous authorities (Section 3).

Fourth, we presented the Sopcos Protocol architecture — comprising Synapse, Axon, and the Core Ledger — as a deployable implementation of this framework, with empirically validated sub-millisecond adjudication latency (Sections 4–5). Fifth, the Cognitive Verdict mechanism establishes a principled boundary between probabilistic AI advisory systems and deterministic safety enforcement, eliminating the Liability Vacuum identified in Section 2.6.

Limitations remain: practical adoption requires gateway integration with legacy ICS infrastructure, trusted key management, and alignment with organizational governance structures. Future work will explore cross-domain standardization, scalability to heterogeneous CPS environments, trusted sensor attestation, and adaptive policy evolution under legal and ethical constraints. Ultimately, by transforming the subjective ambiguity of industrial failure into the objective certainty of cryptographic proof, the Sopcos Protocol establishes the foundational infrastructure required for accountable autonomy in Industry 5.0. It ensures that while machines may be granted the computational authority to act, the moral and legal burden of liability remains explicitly, and provably, human.

References

- [1] European Parliament and Council, “Regulation (EU) 2023/1230 of the European Parliament and of the Council on machinery,” *Official Journal of the European Union*, L 165, pp. 1–102, June 2023.
- [2] International Electrotechnical Commission (IEC), “IEC 62443: Industrial communication networks - Network and system security,” Geneva, Switzerland.
- [3] European Commission, Directorate-General for Research and Innovation, Breque, M., De Nul, L., Petridis, A., *Industry 5.0: towards a sustainable, human-centric and resilient European industry*, Publications Office, 2021.
- [4] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, “Guide to Industrial Control Systems (ICS) Security,” *NIST Special Publication 800-82*, Rev. 2, May 2015.
- [5] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*, pp. 185–200, 2017.
- [6] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum project yellow paper*, 151(2014), pp. 1–32, 2014.
- [7] Sopcos Core Developers, “Sopcos Improvement Proposals (SIP-001 to SIP-021) - Core Architecture and Liability Semantics,” Sopcos Protocol Foundation, 2026.
- [8] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 2, no. 2, pp. 159–176, 1999.
- [9] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” *RFC 6962*, IETF, 2013.
- [10] L. Sweeney et al., “Sigstore: A transparency log for software supply chain security,” *Linux Foundation*, 2021.
- [11] V. Costan and S. Devadas, “Intel SGX Explained,” *Cryptology ePrint Archive*, Report 2016/086, 2016.
- [12] T. Alves and D. Felton, “TrustZone: Integrated Hardware and Software Security,” *ARM White Paper*, 2004.

- [13] D. Lee et al., “Keystone: An Open Framework for Architecting Trusted Execution Environments,” *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020.
- [14] P. T. Husnjak, S. Perakovic, and I. Forenbacher, “Telematics System in Usage Based Motor Insurance,” *Procedia Engineering*, vol. 100, pp. 816–825, 2015.
- [15] J. Lam et al., “IoT-based parametric insurance risk modeling,” *Journal of Risk and Insurance*, 2023.